Python, Machine Learning, Deep Learning & Agentic AI

FINAL

Complete 4-week internship summary, including Week 4: LangChain, LangGraph & AI Agents

1. Introduction

This internship was a structured four-week programme covering the path from Python and mathematical foundations, through classical machine learning and deep learning in PyTorch and TensorFlow, to generative AI, prompt engineering and agentic AI systems built with LangChain and LangGraph. Every week ended with concrete deliverables. This report summarises the full journey and gives detailed coverage of Week 4.

2. Four-Week Overview

Week	Focus	Key Deliverables
1	Python, Mathematics & ML Foundations	Python repo, EDA notebook, Math set, Scikit-learn models
2	Deep Learning with PyTorch	Feed-forward NN, CNN classifier, experiment log, technical report
3	TensorFlow, GenAI & Prompt Engineering	Keras model, CV/NLP project, prompt portfolio, AI-assisted app
4	LangChain, LangGraph & AI Agents	RAG app, LangGraph workflow, LangSmith evaluation, capstone agent

3. Weeks 1 to 3 in Brief

Week 1 - Foundations

Built core Python skills (OOP, data structures, file handling, exceptions), cleaned and explored a real dataset, solved applied maths problems in statistics, probability, linear algebra and calculus, and trained a regression and a classification model with Scikit-learn.

Week 2 - Deep Learning with PyTorch

Implemented a feed-forward network, trained a CNN image classifier, ran controlled hyperparameter experiments on optimizers, learning rates, batch sizes and epochs, and wrote a technical report on the findings.

Week 3 - TensorFlow, GenAI & Prompt Engineering

Built and compared a Keras model with PyTorch, completed a TensorFlow vision or text classification project, produced a prompt engineering portfolio with evaluated iterations, and built and audited an application with AI coding assistance.

Detailed reports for each of these weeks are in Week-1-Report.pdf, Week-2-Report.pdf and Week-3-Report.pdf.

4. Week 4 - LangChain, LangGraph & AI Agents

4.1 RAG Application

ADVANCED

Task Description

Build a LangChain RAG pipeline using FAISS or Chroma.

Objective

Let an LLM answer questions grounded in a private document collection to reduce hallucination.

Approach and Methodology

- Loaded documents and split them into overlapping chunks with a text splitter.
- Created embeddings and stored them in a FAISS or Chroma vector store.
- Built a retriever and a LangChain chain that inserts retrieved context into the prompt.
- Tested with questions, checked that answers were grounded in sources, and tuned chunk size and top-k.
- Documented the architecture: ingestion, indexing, retrieval and generation.

Tools and Libraries

LangChain, FAISS/Chroma, embedding model, LLM API.

Outcome and Results

The RAG pipeline answered questions from the document set with retrieved context. *[Add: your actual result, e.g. dataset name, metric values, screenshots reference]*

Key Learnings

- Chunking and retrieval quality drive answer quality.
- Grounding answers in sources makes them verifiable.

DELIVERABLE Working RAG prototype + architecture notes.

4.2 Multi-Step Workflow in LangGraph

ADVANCED

Task Description

Design a LangGraph multi-step or multi-agent workflow with clear state transitions.

Objective

Model an AI process as an explicit graph so that control flow is predictable and debuggable.

Approach and Methodology

- Defined a shared state schema passed between nodes.
- Implemented nodes (e.g. plan, retrieve, generate, review) and connected them with edges.
- Added conditional edges for branching and loops, such as retry when the review step fails.
- Compiled and ran the graph, inspecting state at each transition, and drew the workflow diagram.

Tools and Libraries

LangGraph, LangChain, LLM API.

Outcome and Results

A multi-step workflow with clear state transitions was implemented and diagrammed. *[Add: your actual result, e.g. dataset name, metric values, screenshots reference]*

Key Learnings

- Explicit state makes agent behaviour easier to control and test.
- Conditional edges enable self-correction loops.

DELIVERABLE LangGraph workflow diagram + code.

4.3 LLM Observability with LangSmith

ADVANCED

Task Description

Use LangSmith to evaluate response quality, latency, and failure cases.

Objective

Monitor and measure LLM application behaviour instead of guessing.

Approach and Methodology

- Enabled LangSmith tracing for the RAG and LangGraph apps.
- Inspected traces for each step: prompts, retrieved context, outputs, token use and latency.
- Created evaluation runs on a test set to assess answer quality and identified failure cases.
- Captured trace screenshots and summarised findings and improvements in an evaluation report.

Tools and Libraries

LangSmith, LangChain, LangGraph.

Outcome and Results

Traces and evaluation runs highlighted latency and failure cases that were then addressed. *[Add: your actual result, e.g. dataset name, metric values, screenshots reference]*

Key Learnings

- Tracing is essential for debugging multi-step LLM systems.
- Evaluation datasets turn subjective quality into measurable results.

DELIVERABLE Evaluation report with trace screenshots.

4.4 CAPSTONE: AI Agent System

ADVANCED | PROJECT

Task Description

Build and document an AI agent integrating RAG, tool use, and multi-step reasoning.

Objective

Combine all internship skills into one deployed agent system.

Approach and Methodology

- Designed the architecture: a LangGraph agent that decides between RAG retrieval and external tools, then reasons over results.
- Implemented tools (e.g. search, calculator or API calls) with clear descriptions so the agent selects them correctly.
- Integrated the RAG retriever and multi-step reasoning with state management and error handling.
- Traced and evaluated the agent with LangSmith, then deployed a live demo.
- Published the GitHub repository with README, wrote the report and prepared presentation slides.

Tools and Libraries

LangChain, LangGraph, LangSmith, FAISS/Chroma, LLM API, GitHub, deployment platform.

Outcome and Results

The capstone agent was built, documented and demonstrated. *[Add: your actual result, e.g. dataset name, metric values, screenshots reference] [Add GitHub and live demo links]*

Key Learnings

- Agents need clear tools, state and guardrails to be reliable.
- Combining retrieval, tools and reasoning produces more capable systems than any single technique.

DELIVERABLE GitHub repo + live demo + report + presentation slides.

5. Skills Gained

- **Programming and data:** Python, OOP, NumPy, Pandas, Matplotlib, Git/GitHub.
- **Mathematics for ML:** statistics, probability, linear algebra, calculus.
- **Machine learning:** Scikit-learn regression and classification, evaluation metrics.
- **Deep learning:** PyTorch and TensorFlow/Keras, CNNs, hyperparameter tuning.
- **Generative AI:** prompt engineering, AI-assisted coding and code auditing.
- **Agentic AI:** RAG, LangChain, LangGraph, LangSmith observability, capstone agent.

6. Challenges and Learnings

- Balancing breadth and depth across many tools in a short time; solved by building each skill on the previous week's work.
- Reliability of LLM outputs; improved through better prompts, grounding with RAG and tracing.
- Reproducibility; addressed with pipelines, fixed seeds, experiment logs and version control.

7. Conclusion

The internship progressed from basic Python to a working AI agent system, with a verifiable deliverable at every step. It provided a strong practical foundation in machine learning, deep learning, generative AI and agentic AI, and a portfolio of projects that can be extended in future work.

Acknowledgement: I thank my mentors and the programme organisers for the structure, guidance and feedback during this internship.